

File: mL_algo_from_scratch/linear_regression.py

Purpose: Linear Regression implementation for continuous value prediction

```
"""Linear Regression (simple, educational implementation).
```

Math Alert: This involves calculus! Derivatives are used to update weights.

Supports closed-form solution (normal equation) and a simple gradient descent.

This file is written plainly with comments to be easy to understand.

```
"""
```

```
import numpy as np
```

=== Imports ===

```
from .utils import add_intercept, mse
```

Math Alert: This involves calculus! Derivatives are used to update weights.

```
from .gradient_descent import GradientDescent
```

=== Classes ===

```
class LinearRegression:
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for '__init__'

```
    def __init__(self, fit_intercept=True):
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
        self.fit_intercept = fit_intercept
```

```
        self.theta = None # parameters (including intercept if used)
```

Logic Focus: Defined task specifically for '_prepare'

```
    def _prepare(self, X):
```

```
        X = np.asarray(X)
```

```
        if X.ndim == 1:
```

```
            X = X.reshape(-1, 1)
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
        if self.fit_intercept:
```

```
            return add_intercept(X)
```

```
        return X
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for 'fit_closed_form'

```
    def fit_closed_form(self, X, y, l2=0.0):
```

```
        """Fit parameters using normal equation (closed-form).
```

```
        l2: ridge regularization strength (simple)
```

```
        """
```

```
        Xp = self._prepare(X)
```

```
        y = np.asarray(y).reshape(-1, 1)
```

```
        d = Xp.shape[1]
```

```
        I = np.eye(d)
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```
        if self.fit_intercept:
```

```
            I[0, 0] = 0.0 # do not regularize bias term
```

```
        A = Xp.T @ Xp + l2 * I
```

```
        self.theta = np.linalg.pinv(A) @ Xp.T @ y
```

```
        self.theta = self.theta.ravel()
```

```
        return self
```

Logic Focus: Defined task specifically for '_grad'

```
    def _grad(self, theta, X, y):
```

```

"""Gradient of MSE loss w.r.t. theta. X already has intercept if needed."""
m = X.shape[0]
preds = X @ theta
grad = (2.0 / m) * (X.T @ (preds - y))
return grad

```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for 'fit_gradient_descent'

```
def fit_gradient_descent(self, X, y, lr=0.01, epochs=1000, tol=1e-8, verbose=False):
```

Math Alert: This involves calculus! Derivatives are used to update weights.

```

"""Fit using simple gradient descent (uses GradientDescent helper)."""
Xp = self._prepare(X)
y = np.asarray(y).reshape(-1)
theta0 = np.zeros(Xp.shape[1])
gd = GradientDescent(lr=lr, epochs=epochs, tol=tol, verbose=verbose)

```

Logic Focus: Defined task specifically for 'grad_fn'

```

def grad_fn(t, Xloc, yloc):
    return self._grad(t, Xloc, yloc)

```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```

theta, history = gd.fit(Xp, y, grad_fn, theta0)
self.theta = theta
return self

```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

Logic Focus: Defined task specifically for 'predict'

```

def predict(self, X):
    Xp = self._prepare(X)
    if self.theta is None:

```

Study Tip: This method usually trains the model. Check how dimensions are handled.

```

        raise ValueError("Model is not fitted yet")
    return Xp @ self.theta

```

Logic Focus: Defined task specifically for 'score'

```
def score(self, X, y):
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```

    preds = self.predict(X)
    return mse(np.asarray(y), preds)

```